

Expt. No: 08
Date: 27.03.2026

MINI PROJECT
SENTIENT SCOUT: AN EMBODIED MISSION AGENT FOR ADAPTIVE FIELD
INTELLIGENCE

AIM:

To develop "Sentient Scout," an edge-AI powered ground vehicle capable of biometric target verification, real-time visual servoing via a 360° pan-tilt mechanism, distance maintenance, and remote tactical deployment via a secure messaging interface for advanced sentry operations.

HARDWARE REQUIRED:

- ❖ Jetson Orin Nano (Edge AI Host Processor)
- ❖ ESP32 (Slave Microcontroller for low-level PID/Motor execution)
- ❖ Camera Module
- ❖ 2-DOF High-Torque TTL Serial Bus Servos (Pan-Tilt Mechanism)
- ❖ 4WD UGV Rover Chassis with Closed-Loop Encoder Motors
- ❖ 3S Lithium Battery UPS System
- ❖ Wi-Fi Module

SOFTWARE REQUIRED:

- ❖ Python 3 & OpenCV (Computer Vision Pipeline)
- ❖ YOLOv8-nano / HOG Descriptor (Lightweight Person Detection)
- ❖ face_recognition / dlib Library (128-Dimensional Facial Embeddings)
- ❖ Custom PID (Proportional-Integral-Derivative) Control Algorithms
- ❖ Telegram Bot API (Remote Command & Alert Interface)
- ❖ JSON Serial Communication Protocol (Jetson to ESP32 interfacing)

THEORY:

Modern surveillance systems are usually static or use open-loop object detection, where algorithms only draw bounding boxes without controlling physical hardware. These systems cannot verify identity autonomously in dynamic environments or maintain optical line-of-sight. Hence, there is a need for an autonomous closed-loop Cyber-Physical System that integrates edge AI with mechanical kinematics.

The **Sentient Scout** system is an embodied AI rover acting as a standalone biometric sentry. The operator manually positions the rover at a chokepoint, uploads a target image via Telegram (e.g., /bounty_mode), and the rover then operates autonomously, detecting, verifying, and tracking the target.

The system uses a cascading logic pipeline to manage computation and hardware limits. The Perception Module continuously captures frames and runs a lightweight person-detection model. To

solve the near-field blind spot, an **Exploratory Tilt** protocol tilts the camera upward when legs or torso are detected to locate the face.

After face detection, the Biometric Verification Module generates 128-dimensional facial embeddings and compares them with the target using Euclidean distance. If a match is found, the system enters active pursuit. The Kinematic Control Module uses Visual Servoing and a PID controller to adjust pan-tilt servos for continuous face tracking.

For depth (Z-axis) tracking, the system monitors the bounding box area. If the target is too close, the rover moves backward; if too far, it moves forward, maintaining the optimal tracking distance without LiDAR. The Alert Module sends real-time images and status updates to the operator via Telegram for remote monitoring.

BLOCK DIAGRAM:

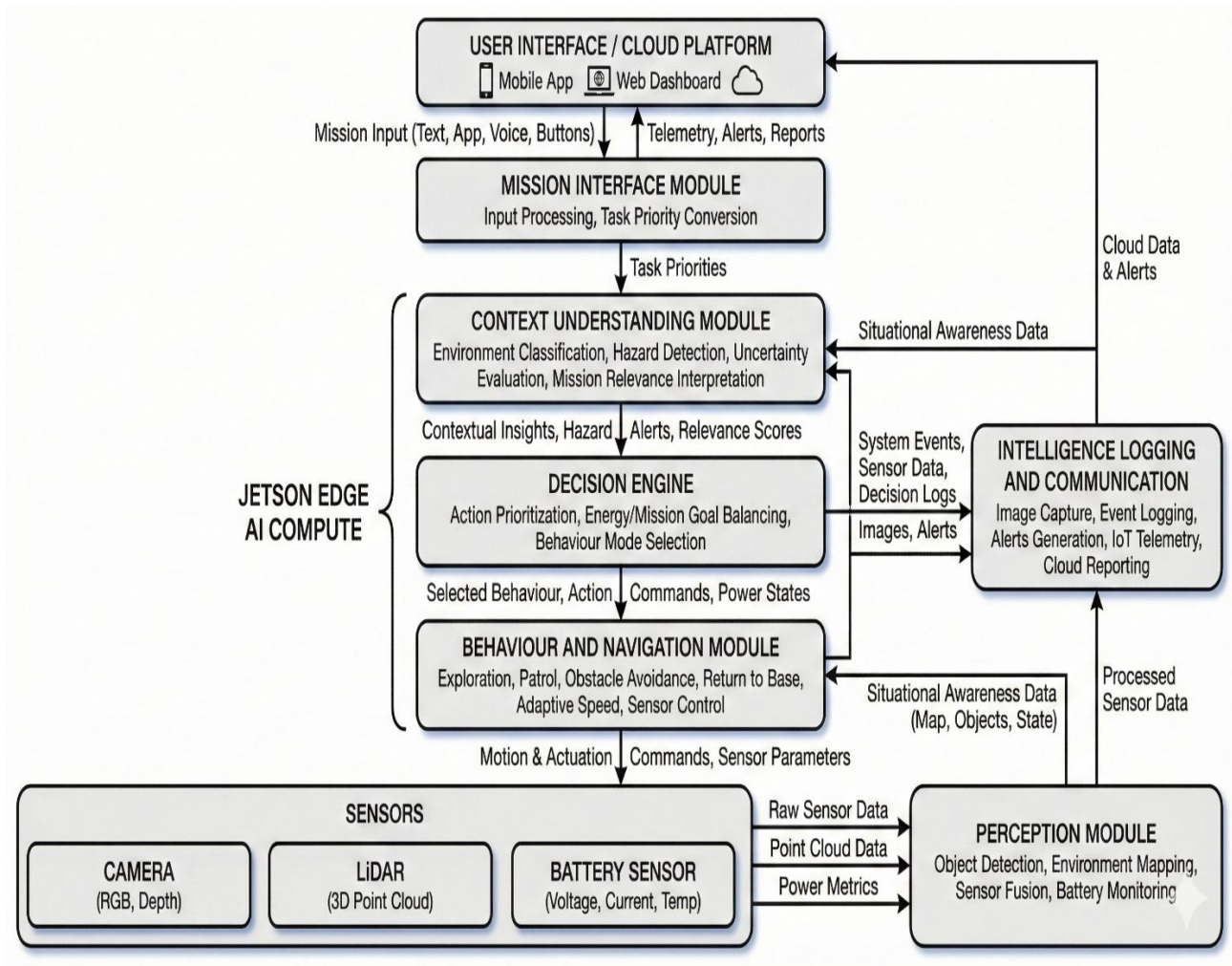


Figure 1. Block diagram for Adaptive field Intelligence (Sentient Scout)

In the remote layer, the system utilizes Wi-Fi to establish a secure communication link. The framework hosts a local Flask web server, allowing the operator to monitor the live, processed video feed with active bounding boxes and identity labels overlaid. For tactical command and critical alerts, the system connects directly to the Telegram Bot API. This bidirectional interface allows the operator to remotely upload target data to initiate the tracking mode, and in turn, transmits instant text alerts and high-resolution captured images back to the operator's device the moment the edge AI confirms a biometric match and acquires a target lock.

ALGORITHM:

1. Start
2. Initialize the rover system, Jetson Orin Nano, camera module, and ESP32 base controller.
3. Receive target biometric data and activation command from operator via Telegram bot.
4. Continuously capture live video frames from the camera module.
5. Run person detection model to check for human presence in the frame.
6. If no human detected, continue scanning and repeat from step 4.
7. If human detected, extract 128-dimensional facial embeddings from the detected face.
8. Compare extracted embeddings against stored target biometric data.
9. If face does not match, continue scanning and repeat from step 4.
10. If face matches, trigger tracking protocol and send alert with captured image to operator via Telegram.
11. Activate pan-tilt servo control to keep target centred in frame.
12. Command ESP32 to drive chassis and maintain optimal following distance.
13. Stream live video feed with bounding box and identity label to Flask web dashboard.
14. Repeat steps 4 to 13 until operator sends stop command.
15. Stop

FLOW CHART:

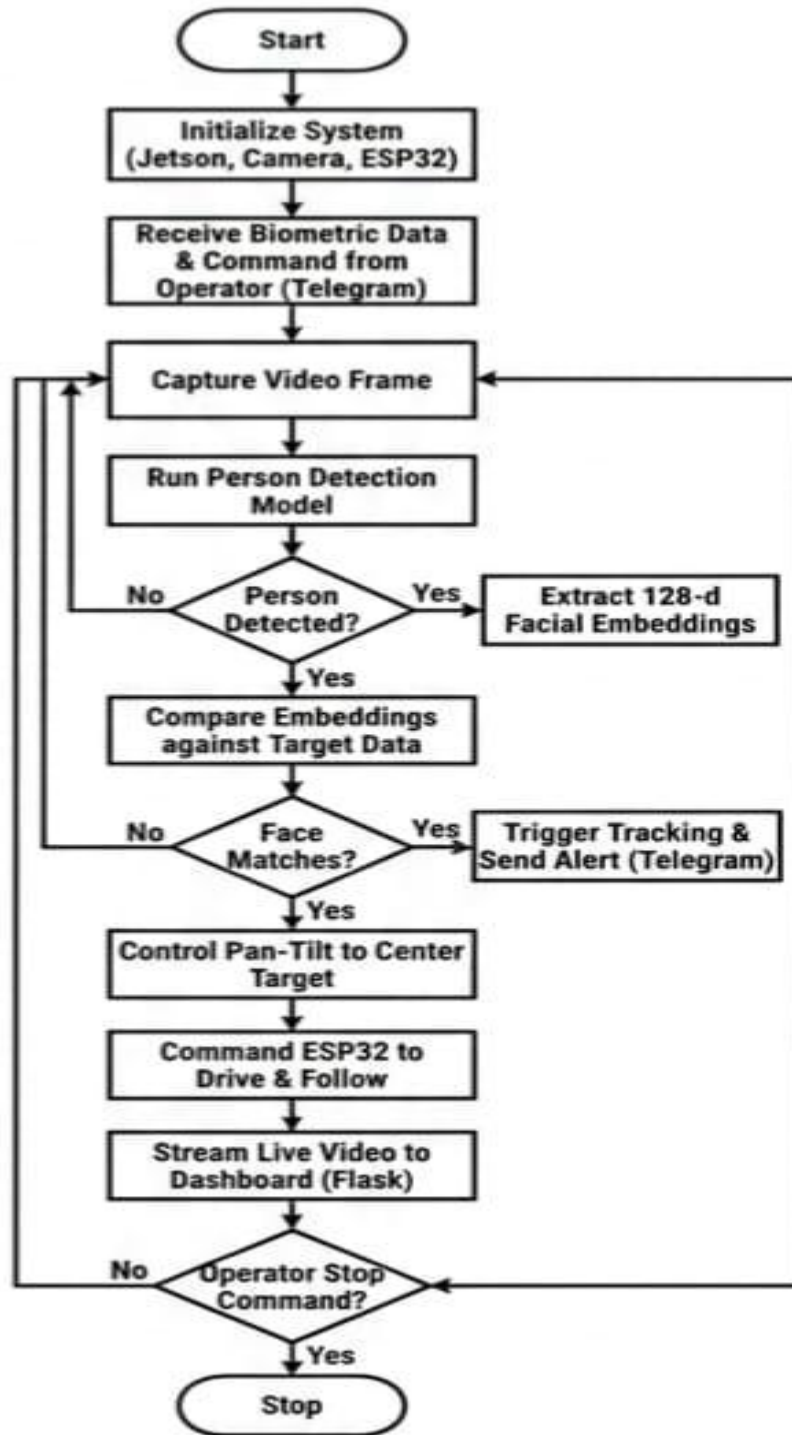


Figure 3. Flow Chart for intrusion detection

CODE:

1) MAIN PROGRAM:

```
import cv2
import threading
import time
import os
import requests
import face_recognition
from flask import Flask, Response
from core.pid import PIDController
from hardware.pantilt import PanTiltMechanism
from config import settings

# --- TELEGRAM SETTINGS ---
TELEGRAM_TOKEN = "8630356500:AAGX9qtCwC52KqXa-pIH-bMArgQ9kFpFJ7E"
CHAT_ID = -1003843758259

app = Flask(__name__)
current_frame = None
thread_lock = threading.Lock()

def send_alert(frame):
    try:
        print("[ALERT] Sending target image to Telegram...")
        _, buffer = cv2.imencode('.jpg', frame)
        url = f"https://api.telegram.org/bot{TELEGRAM_TOKEN}/sendPhoto"
        files = {'photo': ('alert.jpg', buffer.tobytes(), 'image/jpeg')}
        data = {'chat_id': CHAT_ID, 'caption': "SENTRY ALERT: Target identified from floor level."}
        requests.post(url, files=files, data=data, timeout=10)
        print("[ALERT] Telegram message sent successfully.")
    except Exception as e:
        print(f"[ALERT ERROR] Failed to send Telegram message: {e}")

# ASYNCHRONOUS VISION TRACKER

class TestFaceTracker:
    def __init__(self):
        print("[TEST VISION] Initializing Target-Specific Vision...")
        self.cap = cv2.VideoCapture(0, cv2.CAP_V4L2)
        self.cap.set(cv2.CAP_PROP_FRAME_WIDTH, settings.FRAME_WIDTH)
        self.cap.set(cv2.CAP_PROP_FRAME_HEIGHT, settings.FRAME_HEIGHT)
        self.cap.set(cv2.CAP_PROP_BUFFERSIZE, 1)
```

```
self.raw_frame = None
self.running = True

# Thread-safe data storage for the AI's latest findings
self.ai_lock = threading.Lock()
self.target_data = {'is_target': False, 'center': None, 'width': 0, 'bbox': None, 'frame_id': 0}

self.target_encoding = None
if os.path.exists("target.jpeg"):
    image = face_recognition.load_image_file("target.jpeg")
    encodings = face_recognition.face_encodings(image)
    if encodings:
        self.target_encoding = encodings[0]
        print("[TEST VISION] Target encoding successfully loaded.")
    else:
        print("[ERROR] No face found in target.jpeg!")

# Start the decoupled threads
threading.Thread(target=self._camera_thread, daemon=True).start()
threading.Thread(target=self._ai_thread, daemon=True).start()

def _camera_thread(self):
    # THREAD 1: Purely grabs frames as fast as hardware allows (30 FPS)
    while self.running:
        ret, frame = self.cap.read()
        if ret:
            self.raw_frame = frame

def _ai_thread(self):
    # THREAD 2: Heavy AI processing running in the background.
    # Does not block the video feed!
    ai_frame_id = 0
    while self.running:
        if self.raw_frame is None:
            time.sleep(0.01)
            continue

        # Grab a snapshot of the latest frame
        frame = self.raw_frame.copy()
        ai_frame_id += 1

        # --- CLAHE Shadow Recovery ---
        lab = cv2.cvtColor(frame, cv2.COLOR_BGR2LAB)
        l_channel, a_channel, b_channel = cv2.split(lab)
```

```
clahe = cv2.createCLAHE(clipLimit=3.0, tileGridSize=(8,8))
cl = clahe.apply(l_channel)
merged = cv2.merge((cl, a_channel, b_channel))
enhanced_bgr = cv2.cvtColor(merged, cv2.COLOR_LAB2BGR)
rgb_frame = cv2.cvtColor(enhanced_bgr, cv2.COLOR_BGR2RGB)

# Dropped upsample to 1 to save massive processing time
face_locations = face_recognition.face_locations(rgb_frame, model="hog",
number_of_times_to_upsample=1)
face_encodings = face_recognition.face_encodings(rgb_frame, face_locations)

target_found = False

for (top, right, bottom, left), encoding in zip(face_locations, face_encodings):
    face_width = right - left

    if not (15 <= face_width <= 250):
        continue

    if self.target_encoding is not None:
        matches = face_recognition.compare_faces([self.target_encoding], encoding,
tolerance=0.5)
        if True in matches:
            target_found = True
            t_center = (left + (right - left) // 2, top + (bottom - top) // 2)

            # Safely update the global target data
            with self.ai_lock:
                self.target_data = {
                    'is_target': True,
                    'center': t_center,
                    'width': face_width,
                    'bbox': (left, top, right, bottom),
                    'frame_id': ai_frame_id
                }
            break

# If loop finishes and no target was found, clear the data
if not target_found:
    with self.ai_lock:
        self.target_data = {'is_target': False, 'center': None, 'width': 0, 'bbox': None, 'frame_id':
ai_frame_id}

def get_current_state(self):
```

```
# Grabs the 30fps video frame and overlays the latest background AI data
if self.raw_frame is None:
    return None, None, False, 0

frame = self.raw_frame.copy()

with self.ai_lock:
    data = self.target_data.copy()

if data['is_target'] and data['bbox']:
    left, top, right, bottom = data['bbox']
    cv2.rectangle(frame, (left, top), (right, bottom), (64, 128, 255), 2)
    cv2.putText(frame, f"TARGET ( {data['width']}px)", (left, top-10),
                cv2.FONT_HERSHEY_SIMPLEX, 0.6, (64, 128, 255), 2)

    return frame, data['center'], data['is_target'], data['frame_id']

def cleanup(self):
    self.running = False
    if self.cap.isOpened():
        self.cap.release()

def tracking_test_loop():
    global current_frame
    print("--- STARTING WEB-BASED TRACKING TEST ---")

    vision = TestFaceTracker()
    time.sleep(1.5) # Let the camera and AI threads spin up

    servos = PanTiltMechanism()
    pan_pid = PIDController(settings.PAN_P, settings.PAN_I, settings.PAN_D)
    tilt_pid = PIDController(settings.TILT_P, settings.TILT_I, settings.TILT_D)

    last_alert_time = 0
    last_processed_ai_id = -1 # Keeps track of which AI frame the PID reacted to

    while True:
        # This loop now runs blazing fast, pulling the 30FPS feed
        frame, center, is_target, ai_frame_id = vision.get_current_state()

        if frame is None:
            continue

        if is_target and center:
```

```
if time.time() - last_alert_time > 60:
    threading.Thread(target=send_alert, args=(frame.copy(),)).start()
    last_alert_time = time.time()

# --- LATENCY OVERSHOOT FIX ---
# ONLY move the servos if the AI thread has provided a brand new coordinate.
if ai_frame_id != last_processed_ai_id:
    error_x = center[0] - (settings.FRAME_WIDTH // 2)
    error_y = center[1] - (settings.FRAME_HEIGHT // 2)

    pan_adjust = pan_pid.update(error_x)
    tilt_adjust = tilt_pid.update(error_y)

    servos.move(pan_adjust, tilt_adjust)
    last_processed_ai_id = ai_frame_id

# Debug Drawing (runs at 30fps)
cx, cy = settings.FRAME_WIDTH // 2, settings.FRAME_HEIGHT // 2
cv2.drawMarker(frame, (cx, cy), (0, 255, 0), cv2.MARKER_CROSS, 20, 2)
cv2.putText(frame, f"AI Frame: {ai_frame_id}", (10, 30),
cv2.FONT_HERSHEY_SIMPLEX, 0.7, (0, 255, 255), 2)

else:
    # Target is lost. Reset PID memory so it doesn't build up windup,
    # but DO NOT move the servos back to 0,0. Leave them right where they are.
    pan_pid.reset()
    tilt_pid.reset()
    cv2.putText(frame, "NO TARGET", (10, 30), cv2.FONT_HERSHEY_SIMPLEX, 0.7, (0, 0,
255), 2)

with thread_lock:
    current_frame = frame.copy()

# Give the CPU a tiny 10ms breather so the stream doesn't crash
time.sleep(0.01)

def gen():
    while True:
        with thread_lock:
            if current_frame is None:
                continue
            _, buffer = cv2.imencode('.jpg', current_frame, [cv2.IMWRITE_JPEG_QUALITY, 60])
            yield (b'--frame\r\nContent-Type: image/jpeg\r\n\r\n' + buffer.tobytes() + b'\r\n')
```

```
@app.route('/')
def video_feed():
    return Response(gen(), mimetype='multipart/x-mixed-replace; boundary=frame')

if __name__ == '__main__':
    threading.Thread(target=tracking_test_loop, daemon=True).start()
    app.run(host='0.0.0.0', port=5000, threaded=True, use_reloader=False)
```

2) PAN TITLT:

```
# hardware/pantilt.py
import time
from config.settings import PAN_MIN, PAN_MAX, TILT_MIN, TILT_MAX

try:
    from base_ctrl import BaseController
except Exception as e:
    print(f'CRITICAL WARNING: Failed to load base_ctrl. The actual error is: {e}')
    BaseController = None

class PanTiltMechanism:
    def __init__(self):
        # 1. Connect to the rover's serial base controller
        if BaseController is not None:
            try:
                self.base = BaseController('/dev/ttyTHS1', 115200)
                print("Hardware Initialized: Connected to BaseController on /dev/ttyTHS1")
            except Exception as e:
                print(f'Serial Error: {e}')
                print("HINT: You might need to run the script with 'sudo' for serial port access.")
                self.base = None
        else:
            self.base = None

        # 2. Start at the true center for your hardware
        self.current_pan = 0
        self.current_tilt = 0

        # Move to the center position on startup
        self._send_command(self.current_pan, self.current_tilt)

    def move(self, pan_adjust, tilt_adjust):
        # 1. Apply PID adjustments
```

```
# (If the robot moves the wrong way, change += to -=)
self.current_pan += pan_adjust
self.current_tilt -= tilt_adjust

# 2. Safety Clamps using your config settings!
# This allows negative numbers if your MIN settings are negative.
self.current_pan = max(PAN_MIN, min(PAN_MAX, self.current_pan))
self.current_tilt = max(TILT_MIN, min(TILT_MAX, self.current_tilt))

# 3. Send command to hardware
self._send_command(self.current_pan, self.current_tilt)

def _send_command(self, pan, tilt):
    # Print for debugging
    print(f'Executing Move -> PAN: {int(pan)} | TILT: {int(tilt)}')

    # Send physical command over UART
    if self.base is not None:
        # gimbal_ctrl takes (x_angle, y_angle, speed, acceleration)
        self.base.gimbal_ctrl(int(pan), int(tilt), 0, 0)
```

3) PID:

```
# config/settings.py
# Camera Settings
FRAME_WIDTH = 640
FRAME_HEIGHT = 480
CENTER_X = 320
CENTER_Y = 240
# PID Controller Tuning (Start small to prevent violent shaking)
# Lower these numbers for smoother, slower tracking
PAN_P = 0.012 # Was 0.05
PAN_I = 0.0
PAN_D = 0.005

TILT_P = 0.012 # CHANGED FROM 0.0: Must be > 0 for vertical tracking to work
TILT_I = 0.001
TILT_D = 0.008

# Servo Limits (To prevent physical damage to the hardware)
# CHANGED: Shifted to allow movement around the 0,0 center point
PAN_MIN, PAN_MAX = -90, 90
TILT_MIN, TILT_MAX = -90, 90
```

RESULT & DISCUSSION:

The Sentient Scout embodied AI rover successfully demonstrated autonomous target detection, facial recognition, and adaptive tracking. The rover was able to receive the target biometric data from the operator, continuously scan the environment using the camera module, and perform real-time facial embedding extraction and comparison locally on the Raspberry Pi without any cloud dependency.

Upon confirming a target face match, the system successfully initiated the tracking protocol. The pan-tilt servo control kept the target centred in the frame while the ESP32 base controller managed chassis movement to maintain optimal following distance. The rover responded accurately to target movement, adjusting both camera angle and rover direction in real time.

The Telegram Bot successfully delivered instant alerts along with captured images to the operator's device at the moment of target acquisition. The Flask web server provided a continuous live video feed with bounding boxes and identity labels overlaid, allowing the operator to monitor the situation remotely at all times.

This system demonstrates the concept of embodied AI where perception, biometric verification, physical tracking, and remote communication are integrated into a single autonomous rover capable of real-world surveillance and security operations.

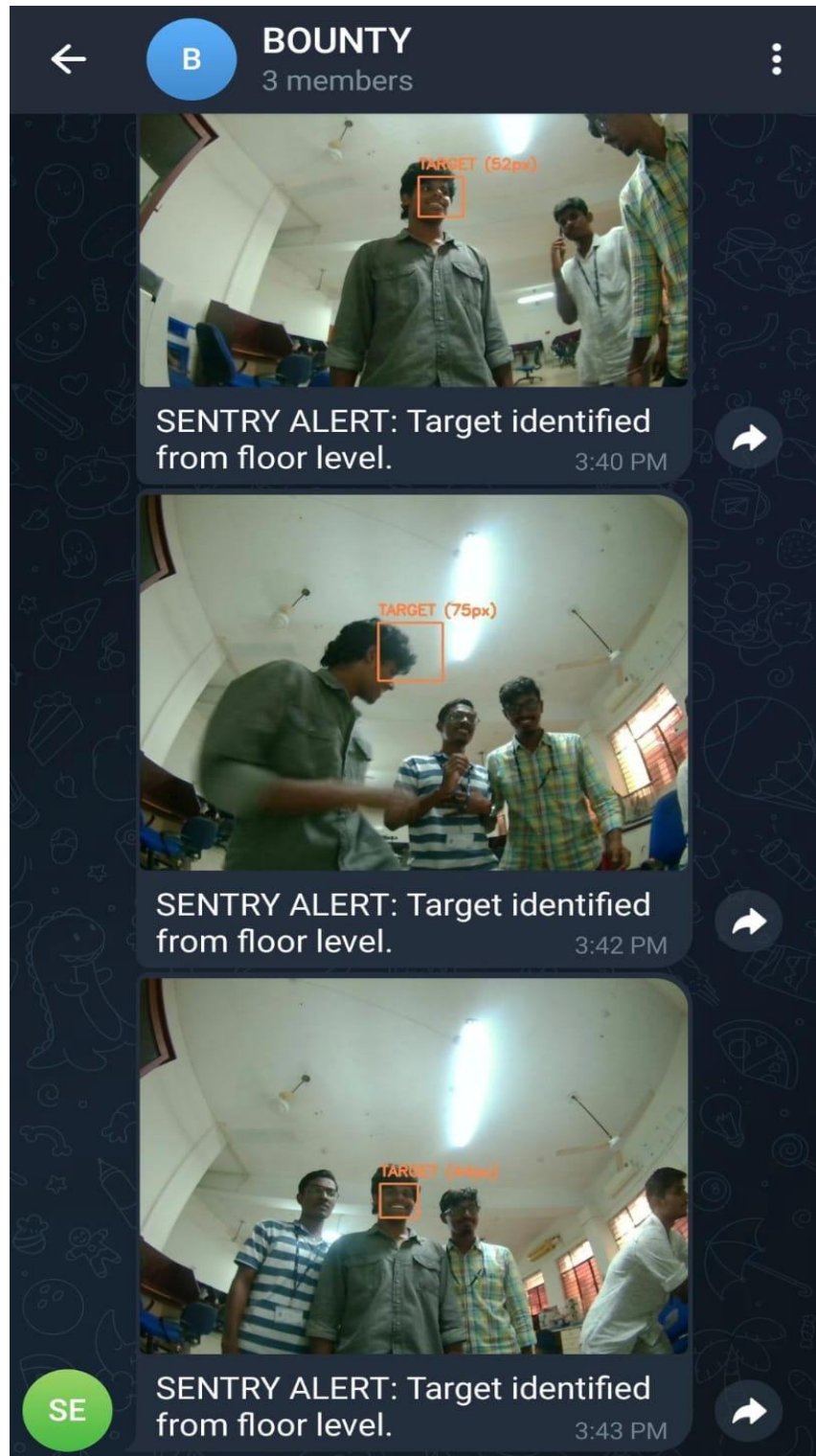


Figure 5. Alert message in Telegram



Figure 6a & 6b. Particular Target detected by Jetson Rover

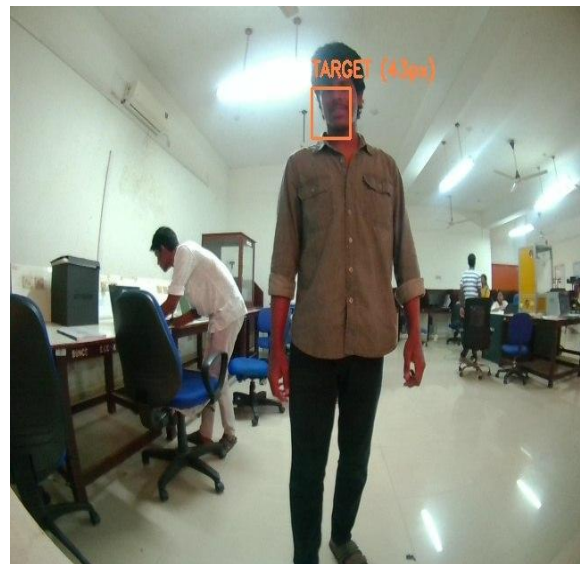
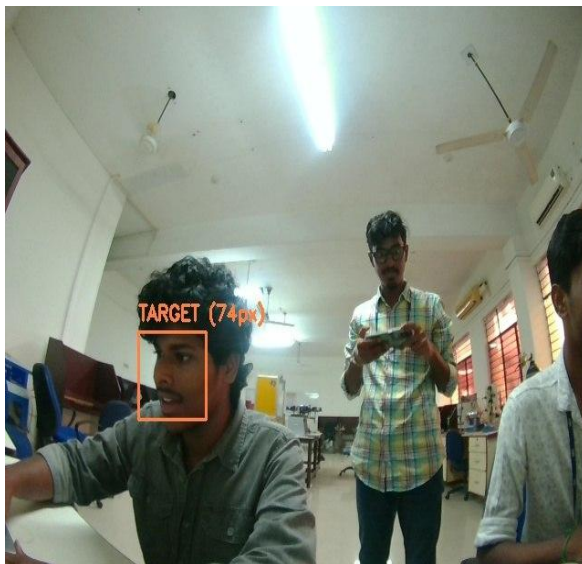


Figure 6c & 6d. Particular Target detected by Jetson Rover



	Design and conduct of experiments	Observation/ Coding/ Implementation	Results	Viva-Voce	Record	Total Marks	Faculty Signature
Maximum Marks	6	6	6	6	6	30	
Allotted Marks							